İZMİR EKONOMİ ÜNİVERSİTESİ

Design Document for “Automated Assignment Evaluation System”

Abdulhamid Yıldırım - 20230602075

Ahmet Emir Doğan - 20220602026

Ario Bashiri - 20230602094

Ayşenaz Gelen - 20230602027

Furkan Pala - 20230602055

Sine Öykü Yaşar - 20230602074

Section: 2

03.05.2026

Table of Contents

1 Introduction.....2
2 Software Architecture.....3
2.1 Structural Design.....3
2.1.1 Model Layer.....5
2.1.2 Logic Layer.....6
2.1.3 UI Layer.....7
2.1.4 Database Schema.....8
2.2 Behavioural Design.....9
2.2.1 Configuration Management Logic.....9
2.2.2 ZIP Extraction and Submission Discovery.....9
2.2.3 Compilation and Execution Pipeline.....9
2.2.4 Output Comparison Logic.....10
2.2.5 Persistent Storage of Projects and Results.....10
2.2.6 Error Handling and Timeout Mechanism.....11
2.2.7 PipeLine Execution Flow.....12
3 Graphical User Interface.....13
3.1 Main Window Layout.....13
3.2 Interactions and Dialogs.....14
3.2.1 New Project Dialog.....14
3.2.2 Open Project Dialog.....15
3.2.3 Import / Export Configuration Dialog.....15
3.2.4 Configuration Dialog.....16
3.2.5 Progress Dialog.....17
3.2.6 Test Case Dialog.....17
3.2.7 Analytics View.....18
3.2.8 Settings View.....18
3.2.9 Results View.....18
3.2.10 Help / About Dialog.....19
4 Deployment.....20

1 Introduction

The Integrated Assignment Environment (IAE) is a standalone desktop application designed to automate the evaluation of programming assignments submitted by students. The application targets Windows desktops and is built using the Java programming language with JavaFX as the graphical user interface framework. Project data is persisted using SQLite, a file-based relational database that requires no server installation.

The IAE allows a lecturer to define language-specific **configurations** (e.g., how to compile and run a C program or how to interpret a Python script), create **projects** that link a configuration with a set of test cases and student submissions, and automatically process each submitted ZIP file through a pipeline of extraction, compilation or interpretation, execution, and output comparison. The results are stored and displayed within the application.

This document describes the software architecture of the IAE. The goal is for a developer to be able to implement the system based solely on this document.

This design satisfies the following requirements from the project specification:

Requirement 1: The software must be deployed to target Windows computers with an installer.

Requirement 2: The software must have a manual accessible via a Help menu.

Requirement 3: The user must be able to create a project using an existing or new configuration.

Requirement 4: The user must be able to create, edit and remove a configuration.

Requirement 5: The user must be able to import and export configurations.

Requirement 6: The software must be able to process ZIP files for each student automatically.

Requirement 7: The software must be able to compile or interpret source code.

Requirement 8: The software must compare student output with expected output.

Requirement 9: The software must display results for each student.

Requirement 10: The user must be able to open and save projects at any time.

2 Software Architecture

The IAE follows a three-layered modular architecture: Model, Logic, and UI. The Model layer defines the core data entities. The Logic layer contains all processing, persistence, and pipeline orchestration. The UI layer handles all user interaction via JavaFX controllers.

2.1 Structural Design

The structural design of the IAE follows a layered architecture organised into three Java packages: “com.iae.model”, “com.iae.logic”, and “com.iae.ui”. The model layer contains pure domain entities representing the persistent data structures of the system. The logic layer contains the evaluation engine, manager classes, and utilities responsible for orchestrating the grading pipeline. The UI layer provides the JavaFX-based graphical interface. Dependencies flow strictly downward: UI depends on logic and model; logic depends on model; model is independent of all other layers. This separation ensures that core grading functionality can be tested without UI components and that UI changes do not propagate into business logic. Since the complete class diagram contains a large number of classes and methods, it is presented in three separate sections for readability: Model Layer (Figure 1), Logic Layer (Figure 2), and UI Layer (Figure 3). The classes within each layer and their responsibilities are described in detail in the following subsections.

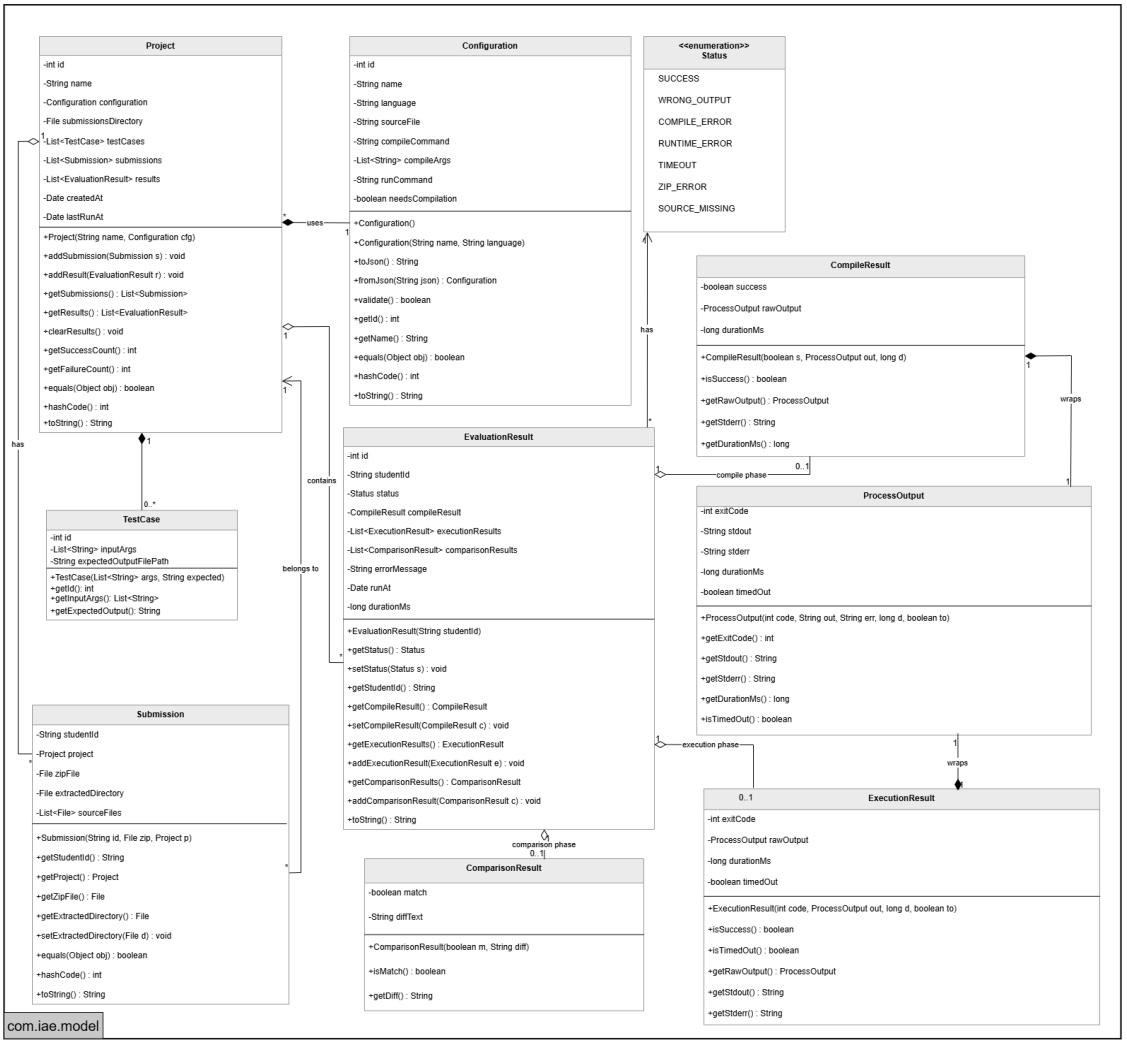

Figure 1: UML (Model Layer)

Figure 1 illustrates the model layer, where Project acts as the central entity connecting Configuration, Submission, and EvaluationResult. The detailed responsibilities of each class are described in Section 2.1.1.

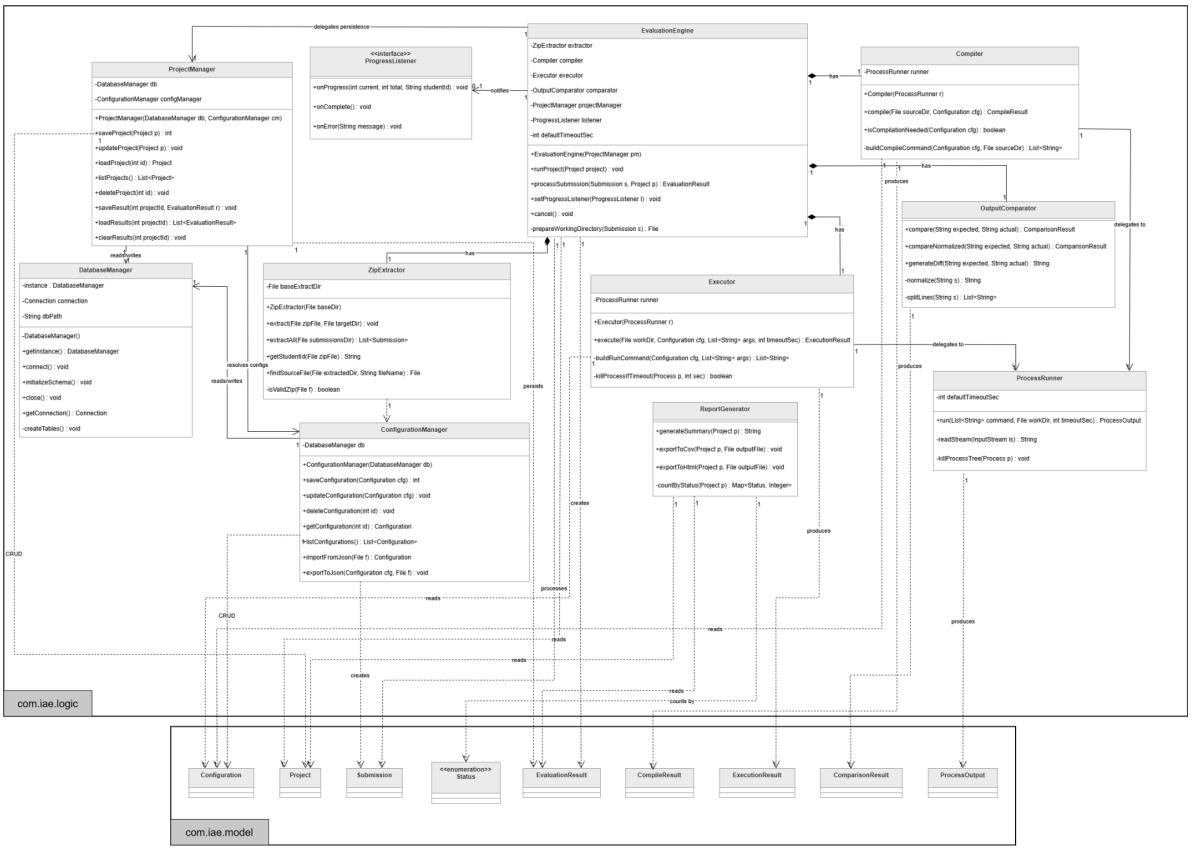

Figure 2: UML (Logic Layer)

Figure 2 shows the logic layer with its dependencies on the model package shown as referenced classes. The EvaluationEngine sits at the centre, orchestrating the components below it. Section 2.1.2 describes each class in detail.

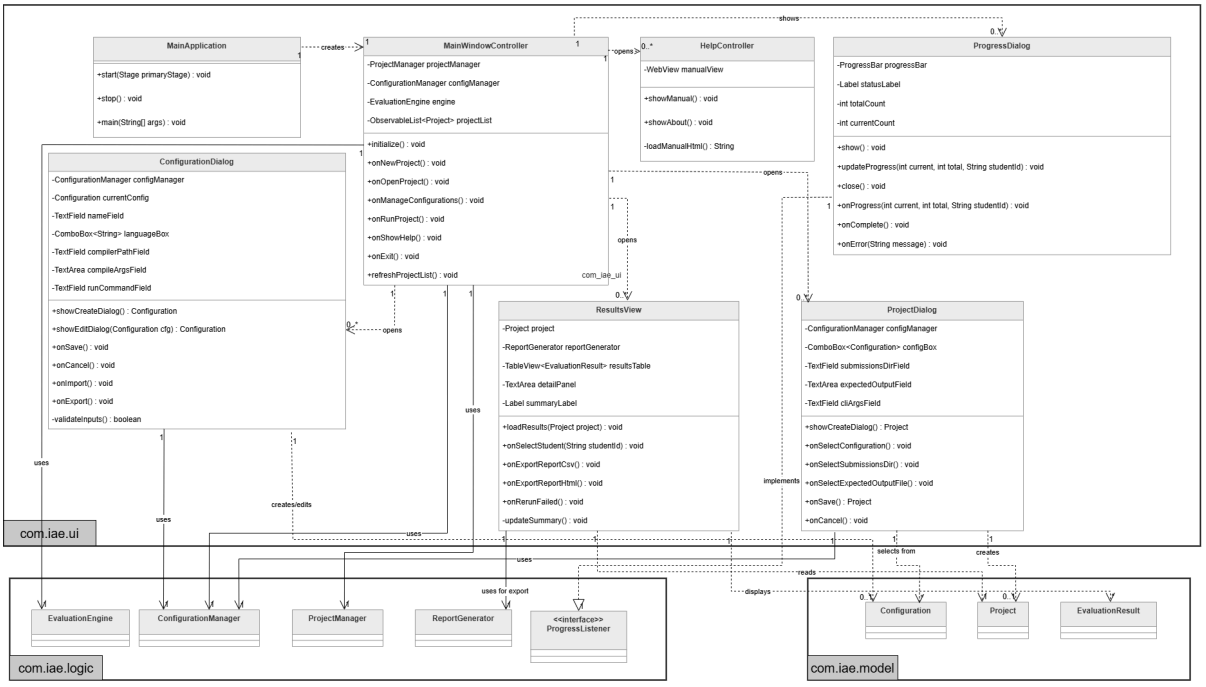

Figure 3: UML (UI Layer)

Figure 3 presents the UI layer, which depends on both the logic and model packages. MainWindowController is the central controller delegating to dialog controllers and the evaluation engine. Section 2.1.3 details each UI component.

2.1.1 Model Layer

The model layer consists of six core classes that represent the domain entities of the system.

Configuration stores everything the system needs to know about a programming language environment. It holds a unique "id", a human-readable "name", a "compileCommand", a list of "compileArgs", a "runCommand", a list of "runArgs", the expected "sourceFileName", and a "needsCompilation" boolean flag. If "needsCompilation" is false, the compile step is skipped. Configurations are serialised to JSON files for import/export **(Requirements 4, 5).**

Project is the central entity that groups everything related to one grading session. It holds a unique "id", a "name", a reference to a "configurationId**"**, a "submissionsDirectory**"** path pointing to the folder containing student ZIP files, and a list of "TestCase" objects that define the test scenarios for the project. Projects are persisted in the SQLite database **(Requirement 3, 10)**.

Submission represents one student's files. It holds a "studentId" (extracted from the ZIP filename), a reference to the parent "Project", a "zipFile" path, an "extractedDirectory" path, and a list of "sourceFiles".

EvaluationResult holds the results for one student after the full pipeline has run. It stores a "studentId", a "Status" enumeration (SUCCESS, WRONG_OUTPUT, COMPILE_ERROR, vb.), a wrapped "CompileResult", a wrapped "ExecutionResult", a wrapped "ComparisonResult", and an "errorMessage". **(Requirement 9)**

TestCase represents a single test scenario defined within a project. It holds a unique "id", a projectId referencing the parent "Project", an "inputArgs" string containing the command-line arguments to be passed to the student's program for this specific test, and an "expectedOutputFilePath" string pointing to the file that contains the expected output for this test. A project may contain one or more "TestCase" objects; the pipeline executes each student's program once per test case and records a separate PASS or FAIL result for each. "TestCase" objects are persisted in the "TESTCASES" table of the SQLite database**.** **(Requirements 3, 8)**

CompileResult, ExecutionResult, and ComparisonResult are immutable value objects that capture the outcome of each pipeline stage. CompileResult holds a success flag and the wrapped raw process output. ExecutionResult holds an exit code, the raw output, and a timeout flag. ComparisonResult holds a match boolean and an optional diff text. ProcessOutput is a low-level container holding stdout, stderr, exit code, duration, and timeout flag, wrapped by both CompileResult and ExecutionResult.

2.1.2 Logic Layer

DatabaseManager follows the Singleton design pattern and is the single point of access to the SQLite database. It manages the Projects, TestCases, and Results tables. It exposes the following methods: "saveProject(Project), loadProject(int id), getAllProjects(), saveTestCasesForProject(List<TestCase>), getTestCasesForProject(int projectId), saveResult(EvaluationResult), and getResultsForProject(int projectId)". On first launch, it creates the database schema automatically.

ProjectManager handles all CRUD operations for projects. It provides createProject(Project), updateProject(Project), deleteProject(int id), loadProject(int id), and getAllProjects() methods. Internally it delegates database operations to DatabaseManager and resolves the linked configuration via ConfigurationManager. (Requirements 3, 10)

ConfigurationManager handles all CRUD operations for configurations and their import/export. Configurations are stored as JSON files in a dedicated configs/ directory (not in SQLite) so they can be easily shared. It provides "createConfiguration(Configuration)", "updateConfiguration(Configuration)", "deleteConfiguration(String id)", "importFromFile(String filePath)", and "exportToFile(Configuration, String filePath)" methods **(Requirements 4, 5)**.

ZipExtractor is responsible for extracting student ZIP files. Since "unzip" cannot be assumed to exist on the system, extraction is implemented in pure Java using "java.util.zip". The "extractAll(String submissionsDir)" method scans the directory for all ".zip" files, extracts each into a subdirectory named after the ZIP file (i.e., the student ID), and returns a list of "Submission" objects. If a ZIP is malformed, the error is recorded, and processing continues with the next file **(Requirement 6)**.

Compiler is responsible for compiling student source code based on a given Configuration. The compile(Submission, Configuration) method builds the compile command from the configuration's compileCommand and compileArgs, uses Java's ProcessBuilder to invoke it in the student's extracted directory, captures stdout and stderr, and returns a CompileResult **(Requirement 7).**

Executor is responsible for running the compiled or interpreted program. The execute(Submission, Configuration, TestCase) method runs the program using the test case's inputArgs, captures stdout and stderr, and returns an ExecutionResult. It enforces a timeout via Process.waitFor(timeout, TimeUnit) to prevent infinite loops **(Requirement 7).** Both Compiler and Executor delegate the low-level process management to a shared ProcessRunner utility class.
There is no separate "Executor" class; both compilation and execution are handled entirely within "CodeRunner". The "compile()" method handles the build step, and the "run()" method handles execution for a given test case.

OutputComparator compares the actual output of a student's program against the expected output file of a "TestCase". The "compare(String actualOutput, String expectedOutputFilePath)" method reads the expected output file and performs a line-by-line string comparison, returning "PASS or FAIL". Whitespace normalisation is applied to avoid false negatives from trailing newlines **(Requirement 8).**

EvaluationEngine orchestrates the full grading pipeline for an entire project. Its "execute(Project, Configuration)" method iterates over all student submissions and for each one:

1. calls "ZipExtractor" to extract the ZIP
2. calls "CodeRunner.compile()" if the language requires compilation
3. for each "TestCase" in the project, calls "CodeRunner.run()" using the test case's "inputArgs"
4. calls OutputComparator.compare() against the test case's "expectedOutputFilePath"
5. constructs an "EvaluationResult" containing per-test-case results and saves it via "DatabaseManager"

Errors at any step are caught, recorded in the report, and the pipeline continues to the next student **(Requirements 6, 7, 8, 9).**

ReportManager provides high-level access to reports for UI display. It fetches all "EvaluationResult" objects for a given project from the "DatabaseManager" and formats summary statistics (total students, pass count, fail count, error count).

ProgressListener is a Java interface that decouples the evaluation pipeline from the UI. It declares two methods: "onProgress(int current, int total, String studentId)", called after each student's submission is processed, and onComplete(), called when the entire batch finishes. "ProgressDialog" implements this interface and updates the progress bar and execution log in response to each event. "PipelineExecutor" accepts a "ProgressListener" instance and invokes these methods at the appropriate points during execution, keeping the UI responsive without creating a direct dependency between the logic and UI layers.

2.1.3 UI Layer

MainApp extends "javafx.application.Application" serves as the entry point. It loads the main FXML layout and initialises the primary stage.

MainWindowController manages the main window and delegates to other controllers and logic classes. It contains references to a "ConfigurationManager", "ProjectManager", and "ReportManager".

ConfigurationDialogController handles the create/edit configuration dialogue. It binds form fields to a "Configuration" object and calls "ConfigurationManager" on save.

ProjectDialogController manages project creation and editing, including selecting a configuration and specifying the submissions directory.

TestCaseDialogController allows adding and editing "TestCase" entries within a project.

ResultsViewController displays the "EvaluationResult" list for the active project in a "TableView", showing student ID, compile status, run result per test case, and any errors.

ImportExportDialogController handles configuration file import and export via a file chooser dialogue.

Requirement 3: Satisfied by "Project", "ProjectManager", and "ProjectDialogController".

Requirement 4: Satisfied by “Configuration”, “ConfigurationManager”, and “ConfigurationDialogController”.

Requirement 5: Satisfied by “ConfigurationManager.importFromFile()” and “exportToFile()”.

Requirement 6: Satisfied by “ZipExtractor” using “java.util.zip”.

Requirement 7: Satisfied by “CodeRunner” using “ProcessBuilder”.

Requirement 8: Satisfied by “OutputComparator”.

Requirement 9: Satisfied by “ResultsViewController” and “ReportManager”.

Requirement 10: Satisfied by “DatabaseManager” and “ProjectManager”.

2.1.4 Database Schema

The application persists project and evaluation data using a relational SQLite database. The Entity-Relationship (ER) diagram below illustrates the schema. To facilitate portability, Configurations are not stored in this database but as separate JSON files; therefore, the link between Projects and Configurations is maintained via a soft logical reference rather than a relational foreign key constraint. Each Project's configuration_id field stores the filename (without extension) of the associated configuration JSON file in the configs/ directory, providing a soft logical reference rather than a relational foreign key.

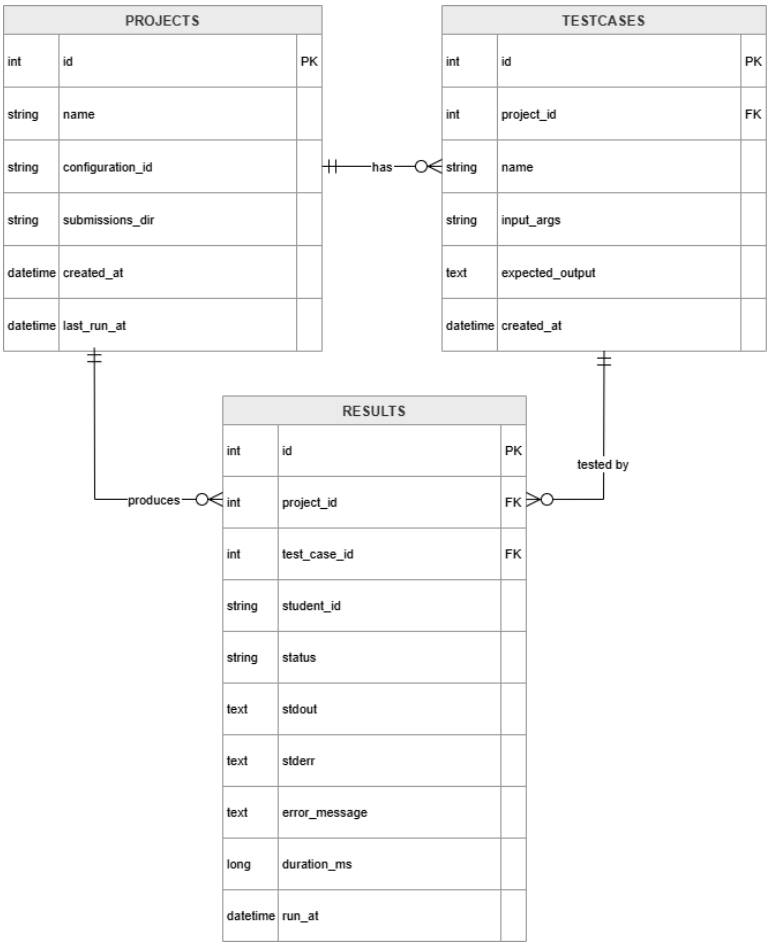

Figure 4: Database Schema

2.2 Behavioural Design

The behavioural design describes how the classes introduced in Section 2.1 work together at runtime to produce the program's intended behaviour. Each subsection below covers one major flow of the application, identifying the classes involved, the order of method invocations, and the conditions under which control branches.

2.2.1 Configuration Management Logic

Configurations are managed entirely through the "ConfigurationManager" class and stored as JSON files within a dedicated "configs/" directory rather than in the SQLite database. When the lecturer opens the Configuration Dialog from the Configurations menu, the "ConfigurationDialogController" presents an empty or pre-filled form and binds each field to a "Configuration" object. On save, the controller uses "ConfigurationManager.createConfiguration()" for new entries or "updateConfiguration()" for edits. The manager serialises the object to JSON and writes it to the "configs/" directory using the configuration's unique id as the filename. Deletion is handled by "deleteConfiguration(String id)", which removes the corresponding JSON file from disk.

Import and export both flow through "importFromFile(String filePath)" which first reads an external JSON file, then deserialises it into a "Configuration" object, later validates the required fields, and lastly copies the file into the "configs/" directory under a freshly assigned id. "exportToFile(Configuration, String filePath)" performs the reverse, which is serialising the selected configuration and writing it to a user-chosen location. As configurations are stored as standalone files, they can be transferred between different computers and can be shared between lecturers if they wish to do so. **(Requirements 4, 5)**.

2.2.2 ZIP Extraction and Submission Discovery

When the lecturer presses the Run button, the "EvaluationEngine" begins by assigning extraction to "ZipExtractor.extractAll(submissionsDir)". The extractor scans the submissions directory for files ending in ".zip", and for each match it derives the student id from the filename, creates a subdirectory of the same name, and extracts the archive's contents using "java.util.zip". The use of the standard Java ZIP API removes any dependency on the host system having an "unzip" utility installed **(Requirement 6)**.

For each archive processed, the extractor constructs a "Submission" object containing the student id, the original ZIP path, the extracted directory path. If a ZIP file is malformed or cannot be opened, the submission's status is set to ERROR, the failure is recorded in the submission's error message, and processing continues with the next file. The method returns a complete list of "Submission" objects to the pipeline.

2.2.3 Compilation and Execution Pipeline

The "EvaluationEngine.runProject()" method drives the per-student grading loop. For each "Submission" returned by the extractor, the status is updated to PROCESSING and the executor consults the configuration's “needsCompilation” flag. When the flag is true, "Compiler.compile(submission, configuration)" is invoked first: the runner constructs a command

from the configuration's "compileCommand" and "compileArgs", uses Java's "ProcessBuilder" to launch the process inside the student's extracted directory, and captures both standard output and standard error along with the process exit code. A non-zero exit code is interpreted as a compile failure — a "EvaluationResult" is built with compileStatus FAILED, the captured error output, and saved via "DatabaseManager", after which the executor proceeds to the next student without attempting to run the program **(Requirement 7)**.

When compilation succeeds, or when compilation is not needed, the executor runs the program once for each TestCase, passing the test case's inputArgs as command-line arguments . The captured program output is then passed to "OutputComparator.compare()" together with the path to the expected output file. The PASS or FAIL result is appended to the student's report. Once all test cases have been processed, the assembled "EvaluationResult" — containing the compile status, per-test-case results, run output, and any error messages — is persisted through "DatabaseManager.saveResult()", and the submission status is marked DONE (Requirements 6, 7, 8, 9).

2.2.4 Output Comparison Logic

The "OutputComparator.compare(String actualOutput, String expectedOutputFilePath)" method reads the expected output file in full and performs a line-by-line comparison against the program's captured standard output. Before comparison, both strings are passed through a whitespace normalisation step that trims trailing spaces from each line and unifies line endings, preventing formatting differences between platforms from causing false negatives. If every normalised line matches, the method returns PASS; otherwise it returns FAIL, optionally accompanied by a diff string identifying the first mismatching line for inclusion in the student's report **(Requirement 8)**.

2.2.5 Persistent Storage of Projects and Results

All persistent project and report data is mediated by the "DatabaseManager" singleton, which holds a single SQLite connection for the lifetime of the application. On the first launch, the manager creates the database file in the application's working directory and initialises the schema — the Projects, TestCases, and Results tables — if they do not already exist. Subsequent launches reuse the existing database without modification.

When the user creates or edits a project, "DatabaseManager.saveProject(Project)" inserts or updates the project row within a single transaction. Loading a project through "loadProject(int id)" retrieves the project row and joins the related evaluation results into a fully reconstructed “Project” object. Each “EvaluationResult” produced by the pipeline is written individually through “saveResult(EvaluationResult)” , and "getReportsForProject(int projectId)" returns the full set for display in the Results View. Because SQLite is file-based, no server is required and the database travels with the application directory **(Requirements 3, 10)**.

2.2.6 Error Handling and Timeout Mechanism

Robustness is enforced at every stage of the pipeline. Timeouts are implemented in "Executor" using "Process.waitFor(timeout, TimeUnit)" — if the student's process does not terminate within the configured limit, "Process.destroyForcibly()" is called and the result is recorded as a TIMEOUT status, preventing infinite loops or runaway programs from blocking the entire batch.

Each pipeline step is wrapped in its own try–catch block so that failures are localised: a malformed ZIP, a missing source file, a compiler crash, a runtime exception, or an unreadable expected-output file are all caught, recorded in the corresponding "EvaluationResult" through its "errorMessage" field, and surfaced to the user in the Results View. Critically, no exception in a single student's processing is allowed to abort the batch — the executor logs the failure and advances to the next submission. The "ProgressDialog" observes the executor through a listener interface, receiving progress events after each student completes, which keeps the UI responsive and updated in real time. A user-initiated cancellation, signalled by the Cancel button on the progress dialog, sets a shared volatile flag that the executor checks between students, allowing the pipeline to halt cleanly at the next safe point **(Requirements 6, 7, 9)**.

The steps of the behavioral pipeline of the program can be summarised using a sequence diagram as given below.

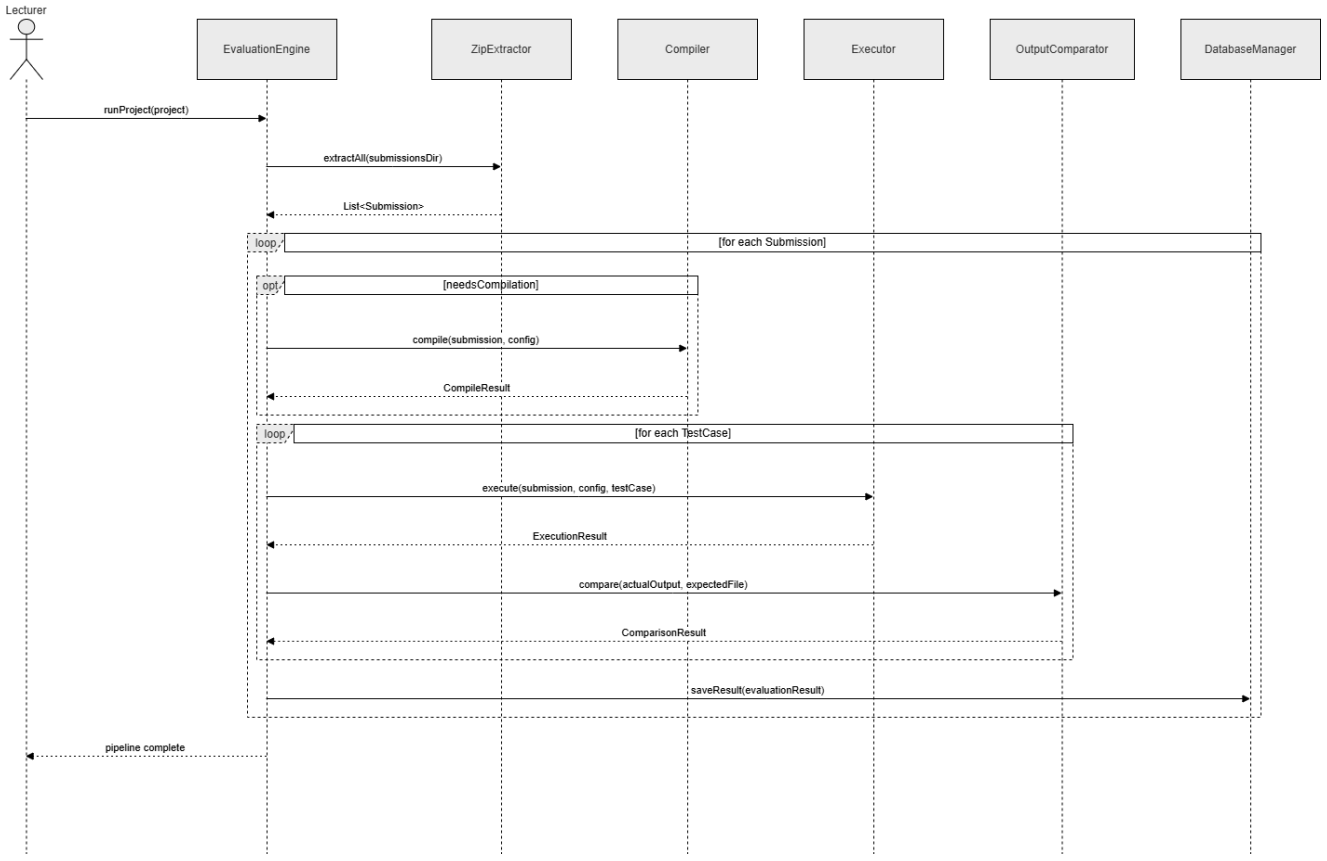

Figure 5: Sequence diagram of the behavioral pipeline.

2.2.7 Pipeline Execution Flow

The activity diagram below illustrates the end-to-end logical flow of the evaluation pipeline once the user initiates a run. It highlights the system's robust, fault-tolerant design; component-level failures are localized. Instead of halting the entire batch, the engine logs the specific error, updates the student's EvaluationResult, and safely proceeds to the next submission. The flow also demonstrates the conditional branching based on the configuration's compilation requirements before proceeding to the execution and output comparison phases.

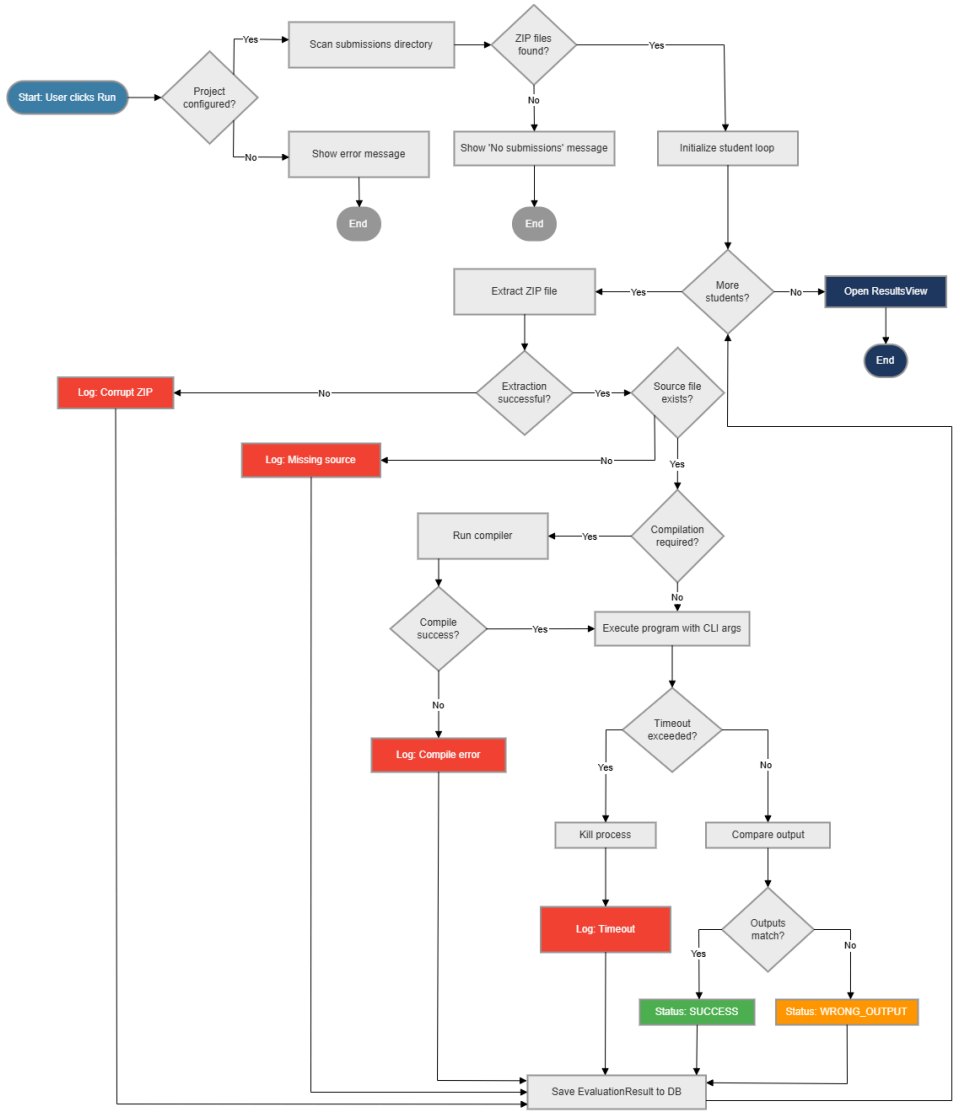

Figure 6: Activity Diagram

3 Graphical User Interface

The IAE's graphical user interface is built with JavaFX and aims for a clean, professional desktop look and feel. All labels, headings, and body text use the Inter typeface, while Cascadia Code is reserved for terminal output and any code-related fields. Content areas are styled in neutral greys and whites, with a primary blue accent that highlights interactive elements like buttons and active states. The window is organised into three persistent regions: a fixed-height menu bar across the top, a collapsible sidebar along the left for navigating between projects, and a main stage that stretches to fill the rest of the available space.

3.1 Main Window Layout

The main window is the primary surface of the application. It is composed of four structural zones.

The **menu bar** spans the full width of the window and contains three menus: **"**File(New Project, Open Project, Save Project)", "Configurations(New, Edit, Remove, Import, Export)", and " Help (Manual, About)". This menu bar satisfies **Requirement 2** (accessible manual) and **Requirement 5** (import/export configurations).

The **Project Explorer sidebar** occupies the left side of the window at a fixed width. It lists all saved projects and provides navigation links: "Projects, Submissions, Analytics, and Settings". Clicking a project entry loads it into the main stage.

The **main stage** displays the currently active project. At the top of the stage, the project name and the active configuration (e.g., *Java Standard 17*) are shown. Below this, a "Submissions Directory" field and a "Browse…" button allow the lecturer to point the application to the folder containing student ZIP files. Below the Submissions Directory field, a "Test Cases" table lists all test cases defined for the project, with columns for "#", "Input Arguments", and "Expected Output File". Each row has "Edit" and "Delete" action buttons. An "+ Add Test Case" button above the table opens the Test Case Dialog, where the user can define a new test case by entering its input arguments and the path to the expected output file **(Requirement 8)**. A prominent "Run" button in the upper-right area of the stage triggers the full grading pipeline **(Requirements 6, 7, 8)**.

The **status bar** at the bottom of the window shows the current application state (e.g., *System Ready*), the number of background tasks in progress, and the active Java environment version.

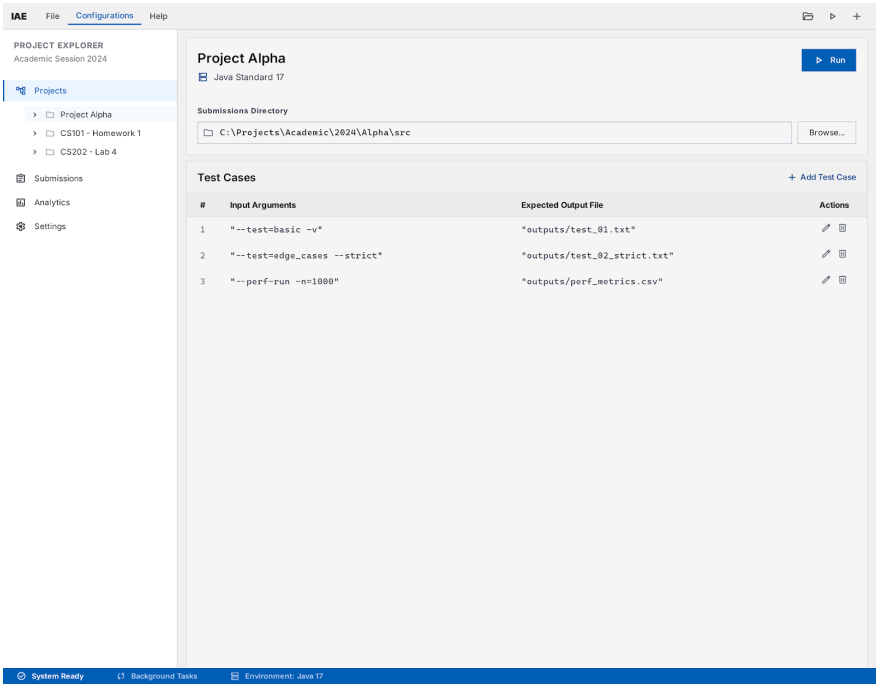

Figure 7: Main Window

3.2 Interactions and Dialogs

3.2.1 New Project Dialog

The New Project Dialog is a modal window that opens when the user selects File → New Project. It contains three fields: a “Project Name” text field, a “Configuration” dropdown populated with all available configurations, and a “Submissions Directory” path field with a “Browse…” button. “Save” and “Cancel” buttons appear at the bottom. On save, a new Project record is created and persisted via **DatabaseManager (Requirements 3, 10)**.

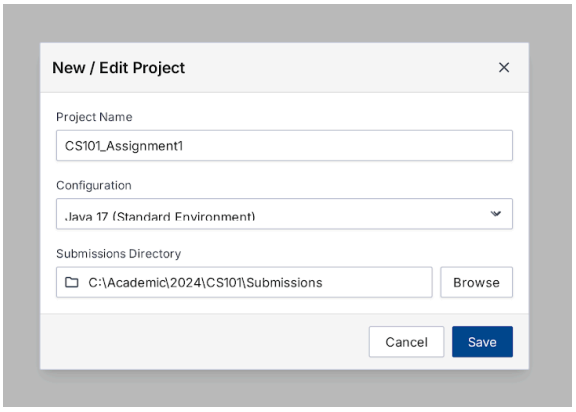

Figure 8: New Project Dialog

3.2.2 Open Project Dialog

Opening an existing project is triggered via **File → Open Project**. Rather than a custom dialog, this action opens the operating system's standard file chooser filtered to ".db" files. The selected file path is passed to "DatabaseManager", which loads the project record and its associated test cases and evaluation results into memory. The loaded project immediately becomes the active project and is displayed in the main stage. If the selected file is not a valid IAE database, an error alert is shown and the current state remains unchanged **(Requirement 10)**.

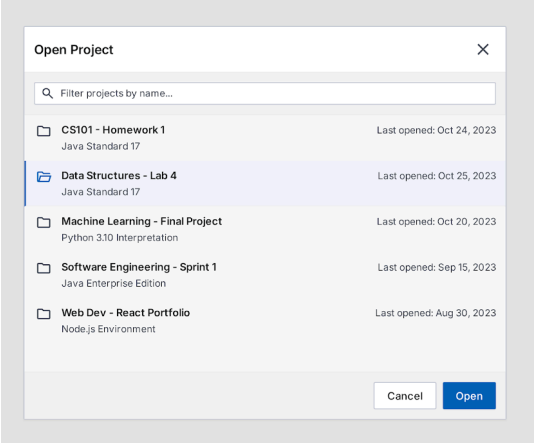

Figure 9: Open Project Dialog

3.2.3 Import / Export Configuration Dialog

Importing a configuration is triggered via **Configurations → Import**. A standard file chooser filtered to ".json" files opens, and the selected file is passed to "ConfigurationManager.importFromFile()" which validates and copies it into the "configs/" directory. The new configuration immediately appears in all configuration dropdowns.

Exporting is triggered via **Configurations → Export** while a configuration is selected. A save-file dialog opens, allowing the lecturer to choose a destination path. "ConfigurationManager.exportToFile()" writes the selected configuration's JSON to that path. Both operations show a brief success or error alert upon completion **(Requirement 5)**.

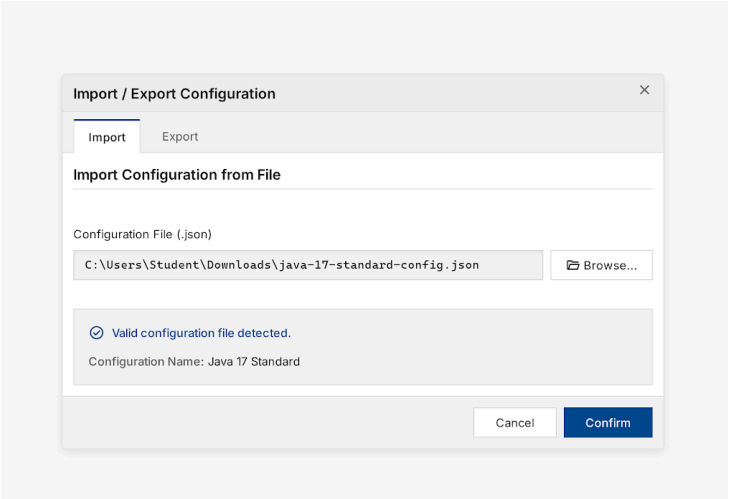

Figure 10: Import/Export Configuration Dialog

3.2.4 Configuration Dialog

The Configuration Dialog opens from Configurations → New or Configurations → Edit. It exposes all fields of the **`Configuration`** model: a "Configuration Name" text input, an Is "Interpreted" toggle that, when enabled, greys out the compile phase fields entirely, a **Compile Phase** section containing the "Compile Command" (e.g. /usr/bin/gcc) and "Compile Arguments" (e.g., main.c -o main) fields, a **Run Phase** section containing the "Run Command" and "Run Arguments" fields, and a **"**Source File Name" field (e.g., main.c or main.py). "Save" and "Cancel" buttons complete the dialog. The greying-out behaviour of the compile fields when "*Is Interpreted"* is checked ensures that interpreted-language configurations cannot accidentally include a compile step **(Requirement 4)**.

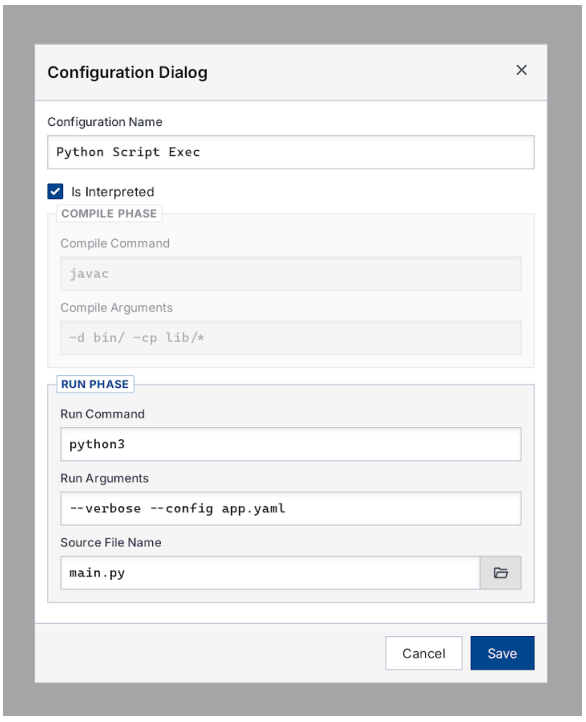

Figure 11: Configuration Dialog

3.2.5 Progress Dialog

When the user presses the "Run ▷" button, the Progress Dialog appears as a non-closable modal overlay. It displays a title (*Processing Submissions*), a subtitle indicating the current student being processed (e.g., *Processing student 4 of 20: s12345*), a horizontal progress bar reflecting the percentage of students completed, and a scrollable **Execution Log** text area. The log grows in real time, recording one line per operation, such as:

[10:01:06] Extracting s12342.zip... OK

[10:01:07] Compiling s12342... OK

[10:01:15] Compiling s12344... FAILED

A Cancel button allows the user to abort the pipeline mid-run. The log text is rendered in a monospace font to preserve alignment. This dialog satisfies **Requirements 6 and 7**.

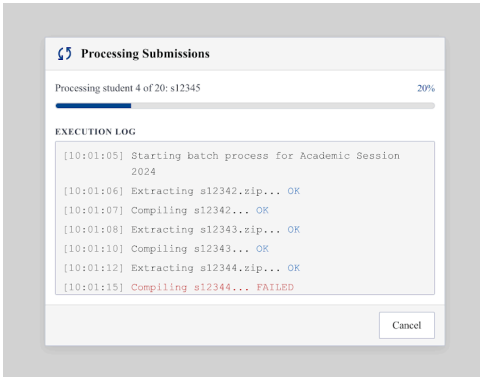

Figure 12: Progress Dialog

3.2.6 Test Case Dialog

The Test Case Dialog is a modal window that opens when the user clicks the + Add Test Case button in the main stage. It contains two fields: an **Input Arguments** text field, where the lecturer enters the command-line arguments to be passed to the student's program (e.g., apple banana cherry), and an **Expected Output File** path field with a "Browse…" button that opens a file chooser. "Save" and "Cancel" buttons complete the dialog. On save, a new **TestCase** object is appended to the active project and persisted via **DatabaseManager (Requirements 3, 8)**.

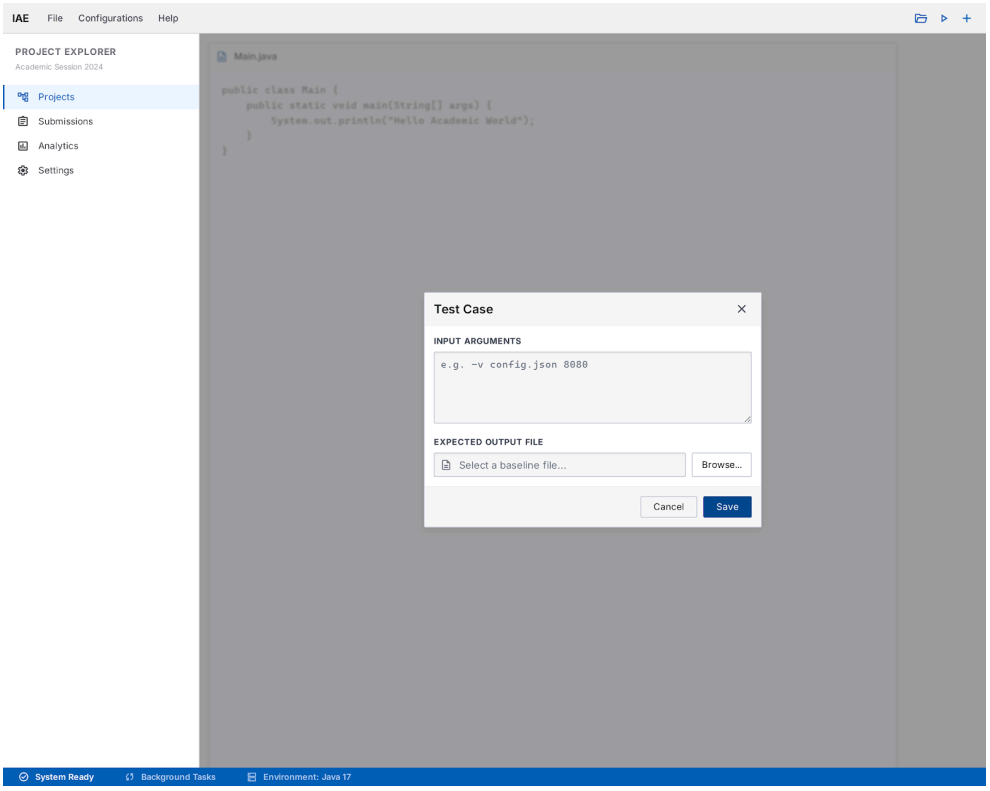

Figure 13:Test Case Dialog

3.2.7 Analytics View

The Analytics View is accessible from the Project Explorer sidebar via the Analytics link. It displays aggregate statistics across all projects. The view contains a summary table showing, for each project, the total number of submissions, pass count, fail count, error count, and pass rate percentage. This view is read-only and refreshes automatically when a new pipeline run completes **(Requirement 9).**

3.2.8 Settings View

The Settings View is accessible from the Project Explorer sidebar via the Settings link. It allows the user to configure application-level preferences, including the default timeout value (in seconds) applied to each student's program execution, and the default submissions directory path. Changes are saved immediately and persist across application restarts.

3.2.9 Results View

Once the pipeline completes, the main stage transitions to the Results View. A summary bar at the top of the stage shows the aggregate statistics for the batch: **Total**, **Pass**, **Fail**, and **Error** counts. Two action buttons **“**Export” and “Re-Run All” appear beside the summary bar.

The core of the Results View is a **TableView** built around several columns: **Student ID**, **Compile Status** (SUCCESS or FAILED), a dynamic set of per-test-case columns labelled TC1, TC2, TC3, … each reporting PASS or FAIL, and finally an **Errors / Output** column that surfaces a short excerpt of the first error or exception logged for that student. To make scan-reading easier, rows are colour-coded — green for a fully passing submission, red where compilation failed, and yellow where the student compiled successfully but lost points on at least one test case. Clicking a row expands a **Detail Output** panel at the bottom of the stage, which renders the complete compiler output or run log for that student in a monospace font **(Requirement 9)**.

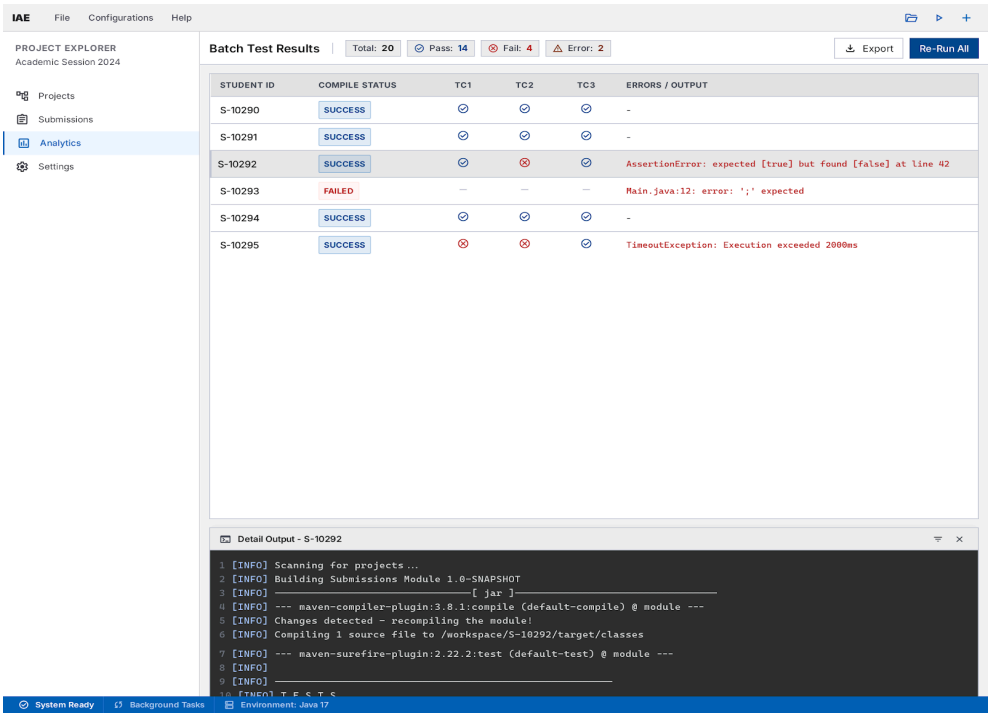

Figure 14: Results View

3.2.10 Help / About Dialog

The Help dialog is accessible from **Help → Manual**. It opens as a resizable window containing a scrollable text area that renders the user manual in formatted prose. The manual covers workspace organisation, core keyboard shortcuts (e.g., Ctrl+N for a new project, F9 to run the pipeline), and guidance on configuring the execution environment. A Close Manual button is provided at the bottom of the window **(Requirement 2)**.

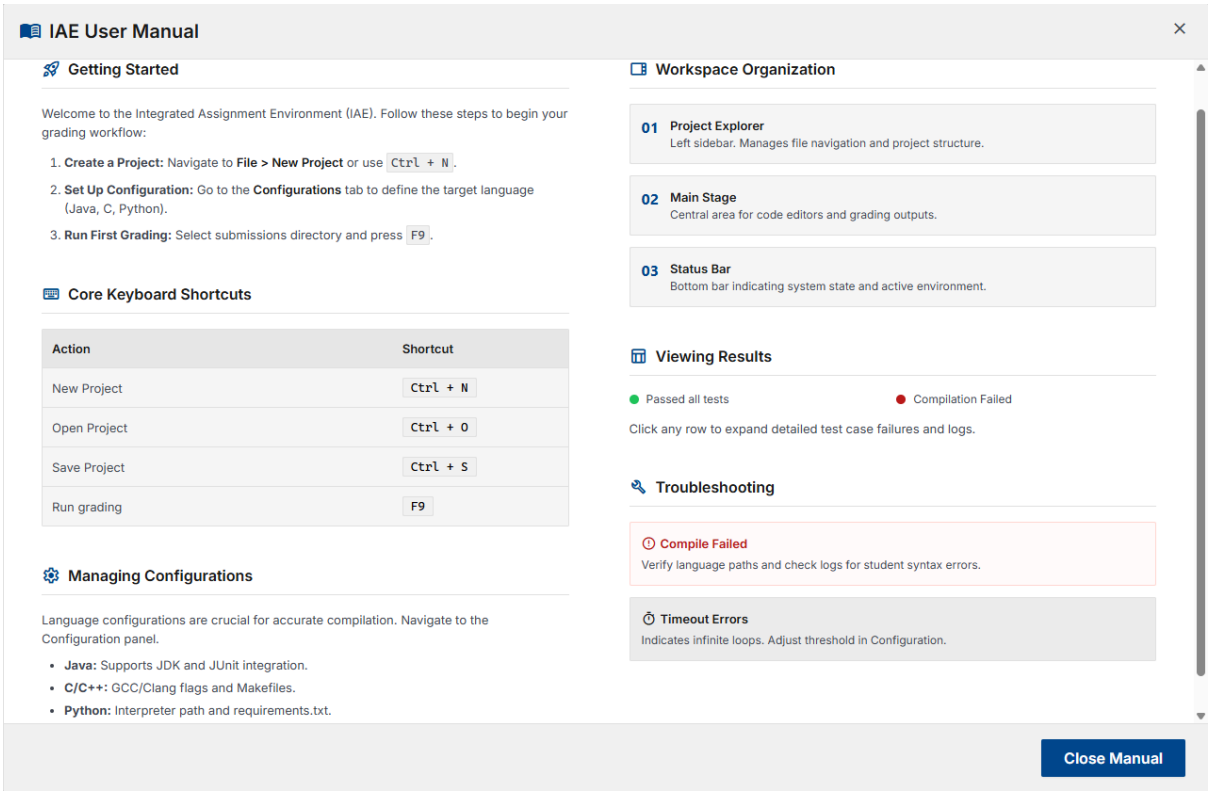

Figure 15: Help / Manual Dialog

4 Deployment

The IAE is distributed as a single-file Windows installer built with **Inno Setup**. The build process compiles the Java source with Maven, packages the result as a fat JAR using the Maven Shade Plugin, and then passes the JAR to Inno Setup to produce a standard Windows ".exe" installer.

The Inno Setup script bundles the following components:

- The fat JAR (iae.jar)
- A bundled JRE (Java Runtime Environment) so the end user does not need Java installed separately
- A "configs/" directory placeholder for storing configuration JSON files
- A start menu shortcut and a desktop icon pointing to the launcher

The installer places all files under "%ProgramFiles%\IAE\" by default and registers an uninstaller entry in Windows. On first launch, "DatabaseManager" creates the SQLite database file (iae.db) in the application's working directory automatically. **(Requirement 1)**